

14.7 Chassis Tracking

In this lesson, OpenCV is used to recognize a target-colored block and control the robot chassis to track it.

Before starting this activity, please prepare a color block of any color.

1. Program Introduction

The program first subscribes to the ROS2 image node image_raw to receive the video feed. By clicking on the image, the user selects the target area, allowing the system to capture and record the color range for target identification. Once the target color is identified, OpenCV calculates the coordinates of its center. The system then determines the offset between this point and the center of the image frame. This offset is passed to a PID controller, which uses predefined parameters to calculate the appropriate movement speed for the chassis. Finally, the control data is published through the chassis control node topic, enabling the robot to track and follow the target accurately.

2. Operation Steps

Note: Commands are case-sensitive, so please enter them exactly as required. You may use the **Tab** key to auto-complete keywords for convenience.

- 1) Start the robot, and access the robot system using the VNC remote control software by referring the instructions in '**8. Remote Tool Installation and Container Access/ 8.1 VNC Installation and Connection**'



- 2) Click-on  to launch the command-line terminal.
- 3) Run the following command to disable the app auto-start service.

```
~/./stop_ros.sh
```

```
> ~/./stop_ros.sh  
zsh: killed ~/./stop_ros.sh
```

- 4) Enter the command below to initiate the game.

```
ros2 launch app object_tracking_node.launch.py debug:=true
```

```
> ros2 launch app object_tracking_node.launch.py debug:=true  
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2025-04-23-21-46-11-018626-raspberrypi-5431  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [ascamera_node-1]: process started with pid [5433]  
[INFO] [static_transform_publisher-2]: process started with pid [5435]  
[INFO] [static_transform_publisher-3]: process started with pid [5437]  
[INFO] [joint_state_publisher-4]: process started with pid [5439]  
[INFO] [robot_state_publisher-5]: process started with pid [5441]  
[INFO] [ros_robot_controller-6]: process started with pid [5443]  
[INFO] [odm_publisher-7]: process started with pid [5445]  
[INFO] [ekf_node-8]: process started with pid [5448]  
[INFO] [object_tracking-9]: process started with pid [5456]  
[ascamera_node-1] [INFO] [1745444771.835802420] [as_logger]: hello world angstrong camera ros2 node
```



- 5) Click  to open a new terminal, then enter the following command to launch the camera. Once the live camera feed appears, click on the target area to select and capture its color.

```
ros2 service call /object_tracking/enter std_srvs/srv/Trigger {}
```

```
ros2 service call /object_tracking/enter std_srvs/srv/Trigger {}
```

- 6) Enter the following command to start tracking.

```
ros2 service call /object_tracking/set_running std_srvs/srv/SetBool data:\ true
```

```
ros2 service call /object_tracking/set_running std_srvs/srv/SetBool  
data:\ true
```

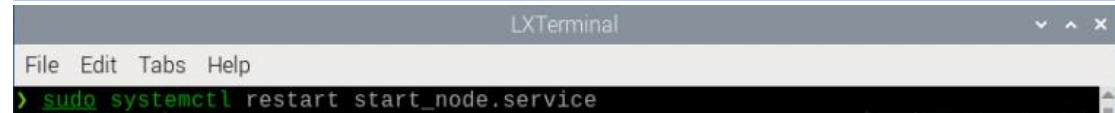
- 7) To exit this mode, press Ctrl+C in the terminal. If it doesn't close successfully, please try again multiple times.

After finishing this activity, you can start the mobile app service either by entering a command or by restarting the robot. If the mobile app service is not activated, app-related features will not function properly. (Note: The

mobile app service will start automatically after a robot reboot.)

- 8) Click  to open a terminal and enter the following command to restart the app service. A beeping sound from the buzzer will indicate that the service has been successfully enabled.

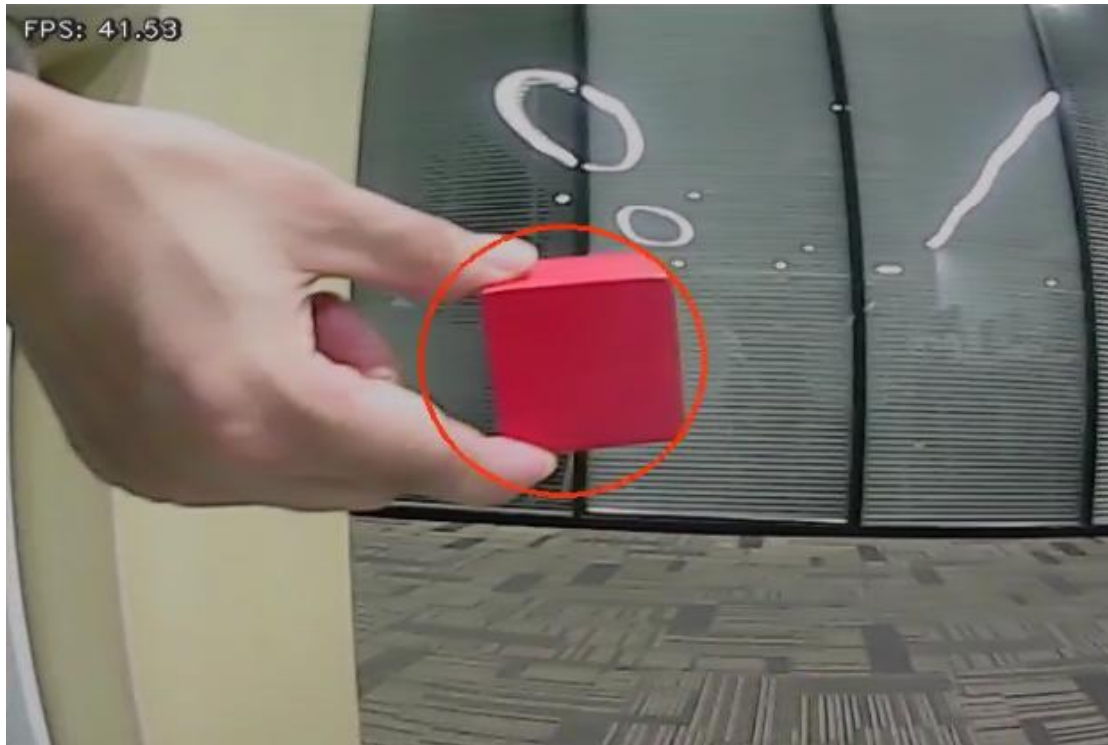
```
sudo systemctl restart start_node.service
```



3. Program Outcome

Note: After starting the game, please ensure that no other objects containing the target color are within the camera's field of view, as this may interfere with proper recognition and tracking.

Once the game starts, place the target object within the camera's view. When the object is detected, it will be marked with a circle, and the robot will begin tracking it accordingly.



4. Program Analysis

Based on the observed behavior, review and organize the program's logical flow as shown in the diagram below.

The source code for this program is located inside the Docker container at:

```
/home/ubuntu/ros2_ws/src/app/app/object_tracking.py
```

- ◆ The main function is responsible for displaying the image, handling mouse events, and publishing chassis movement commands.

```

147 def main(self):
148     while True:
149         try:
150             image = self.image_queue.get(block=True, timeout=1)
151         except queue.Empty:
152             continue
153
154         result = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
155         cv2.imshow("result", result)
156         if self.debug and not self.set_callback:
157             self.set_callback = True
158             # 设置鼠标点击事件的回调函数
159             cv2.setMouseCallback("result", self.mouse_callback)
160         k = cv2.waitKey(1)
161         if k != -1:
162             break
163     self.mecanum_pub.publish(Twist())
164     rclpy.shutdown()

```

◆ Pid calculates the data of the chassis motion

```

77 if circle is not None:
78     self.lost_target_count = 0
79     (x, y), r = circle
80     x = x / self.pro_size[0] * w
81     y = y / self.pro_size[1] * h
82     r = r / self.pro_size[0] * w
83
84     cv2.circle(result_image, (self.x_stop, self.y_stop), 5, (255, 255, 0), -1)
85     result_image = cv2.circle(result_image, (int(x), int(y)), int(r), (self.target_rgb[0],
86     self.target_rgb[1],
87     self.target_rgb[2]), 2)
88
89     vx = 0
90     vw = 0
91     if abs(y - self.y_stop) > 20:
92         self.pid_dist.update(y - self.y_stop)
93         twist.linear.x = common.set_range(self.pid_dist.output, -0.35, 0.35)
94     else:
95         self.pid_dist.clear()
96         if abs(x - self.x_stop) > 20:
97             self.pid_yaw.update(x - self.x_stop)
98             if self.machine_type == 'MentorPi_Acker':
99                 steering_angle = common.set_range(-self.pid_yaw.output, -math.radians(322/2000*180), math.radians(322/2000*180))
100                 if steering_angle != 0:
101                     R = 0.145/math.tan(steering_angle)
102                     twist.angular.z = -twist.linear.x/R
103             else:
104                 twist.angular.z = common.set_range(self.pid_yaw.output, -2, 2)
105         else:
106             self.pid_yaw.clear()
107
108     return result_image, twist

```

The received position data is used to calculate the offset from the target.

A PID controller then computes the appropriate linear and angular velocities for the chassis, which are continuously published in a loop within the main function.

◆ Set Color Threshold

```
95     def set_color_srv_callback(self, request, response):
96         self.get_logger().info('\033[1;32m%s\033[0m' % "set_color")
97         msg = SetColorDetectParam.Request()
98         msg_red = ColorDetect()
99         msg_red.color_name = request.data
100        msg_red.detect_type = 'circle'
101        msg.data = [msg_red]
102        res = self.send_request(self.set_color_client, msg)
103        if res.success:
104            self.get_logger().info('\033[1;32m%s\033[0m' % 'start_track_%s'%msg_red.color_name)
105        else:
106            self.get_logger().info('\033[1;32m%s\033[0m' % 'track_fail')
107        response.success = True
108        response.message = "set_color"
109        return response
110
```

Set the color threshold. Configure the color detection parameters based on the request and publish them accordingly.